

数据结构与算法

第2章 线性表

张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2026年3月20日

上节回顾

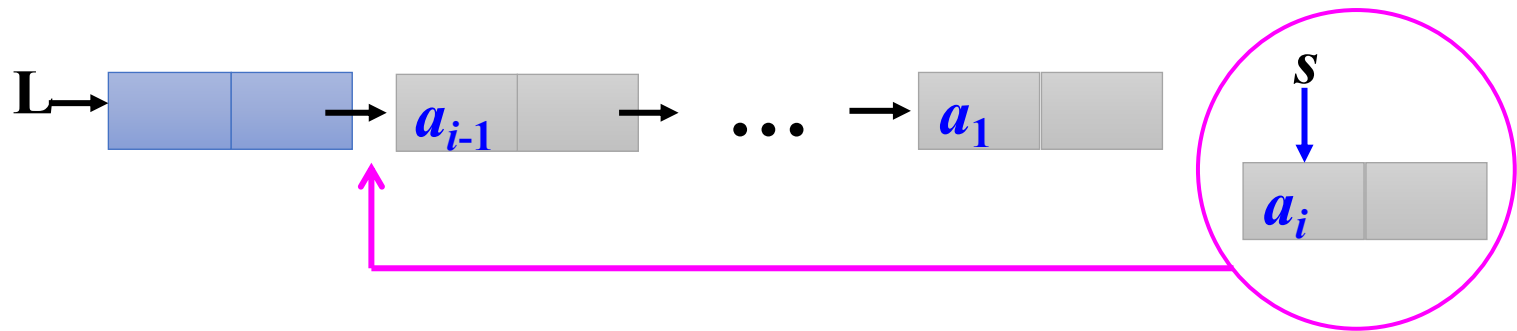
1、单链表的建立

头插法

尾插法

单链表的建立—— (1) 头插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表头上，直到结束为止。



注意：链表的结点顺序与逻辑次序相反。

```
s->next=L->next;    //将s插在原开始结点之前，头结点之后  
L->next=s;
```

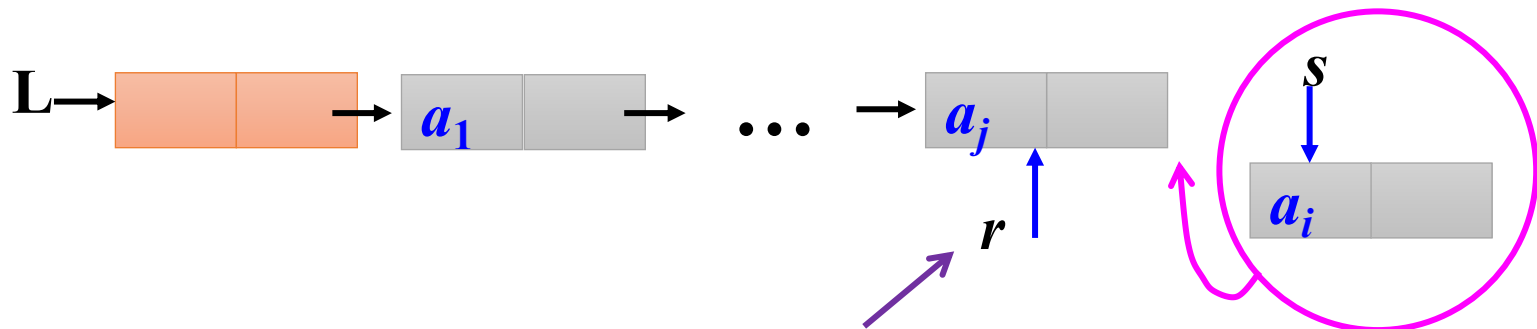
头插法建表算法如下：

```
void CreateListF(LinkNode *&L, ElemType a[], int n)
{ LinkNode *s;
  int i;
  L=(LinkNode *)malloc(sizeof(LinkNode));
  L->next=NULL;  //创建头结点，其next域置为NULL
  for (i=0;i<n;i++)  //循环建立数据结点
  { s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=a[i];  //创建数据结点s
    s->next=L->next; //将s插在原开始结点之前，头结点之后
    L->next=s;
  }
}
```

O(n)

单链表的建立—— (2) 尾插法建表

- 从一个空表开始，创建一个头结点。
- 依次读取字符数组 a 中的元素，生成新结点
- 将新结点插入到当前链表的表尾上，直到结束为止。



增加一个尾指针 r ，使其始终指向当前链表的尾结点

注意：链表的结点顺序与逻辑次序相同。

```
r->next=s;           //将s插入r之后  
r=s;
```

尾插法建表算法如下:

```
void CreateListR(LinkNode *&L, ElemType a[], int n)
{ LinkNode *s, *r;
int i;
L=(LinkNode *)malloc(sizeof(LinkNode)); //创建头结点
r=L; //r始终指向尾结点，开始时指向头结点
for (i=0;i<n;i++) //循环建立数据结点
{
    s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=a[i]; //创建数据结点s
    r->next=s; //将s插入r之后
    r=s;
}
r->next=NULL; //尾结点next域置为NULL
}
```

O(n)

上节回顾

1、单链表的建立

头插法

尾插法

2、双向链表的定义和基本操作

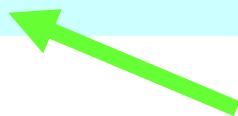
为什么要讨论双向链表？

建立、插入、删除：两个指针域——同时修改

双向链表

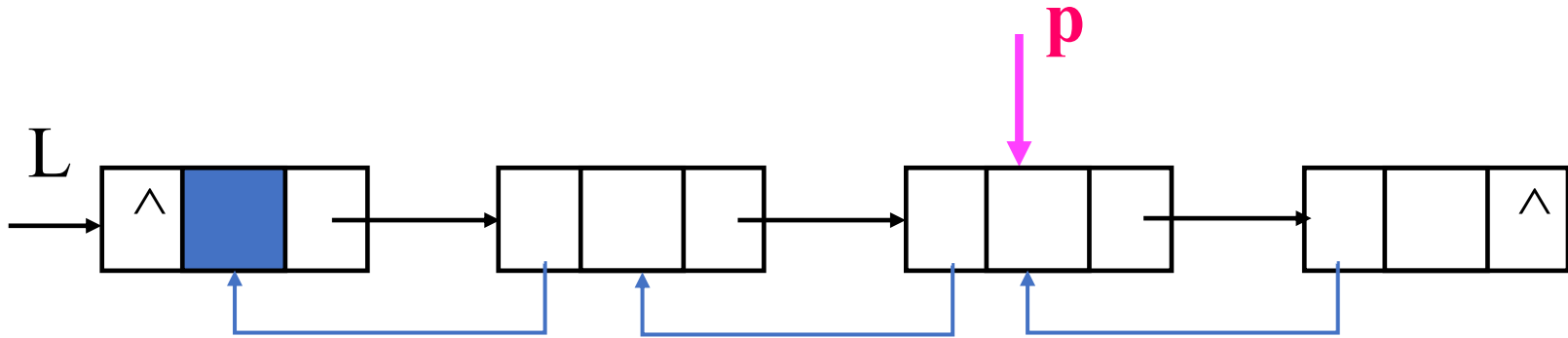


```
typedef struct DNode    //双向链表结点类型
{ ElemType data;
  struct DNode *prior;  //指向前驱结点
  struct DNode *next;   //指向后继结点
} DLinkNode;
```



双向链表结点结构

双向链表结构的对称性（设指针p指向某一结点）



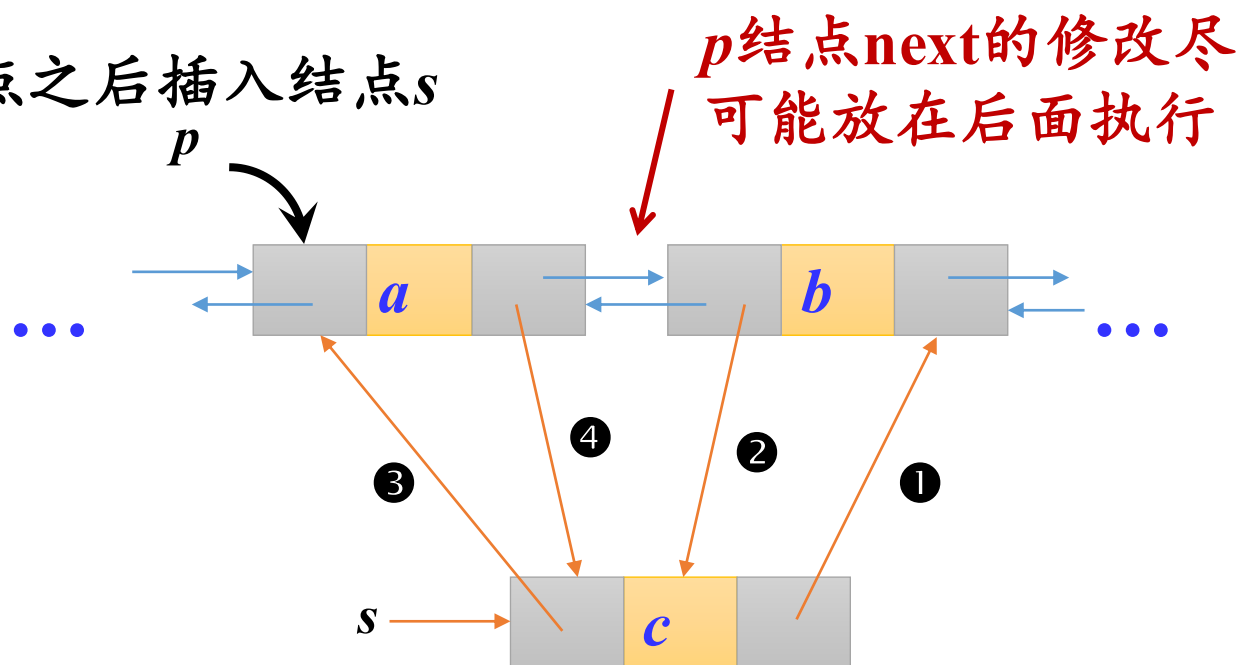
$$p \rightarrow \text{next} \rightarrow \text{prior} = p = p \rightarrow \text{prior} \rightarrow \text{next}$$

双链表的优点：

- 从任一结点出发可以快速找到其前驱结点和后继结点；
- 从任一结点出发可以访问其他结点。

双向链表的插入

在 p 结点之后插入结点 s



操作语句：

① $s \rightarrow \text{next} = p \rightarrow \text{next}$

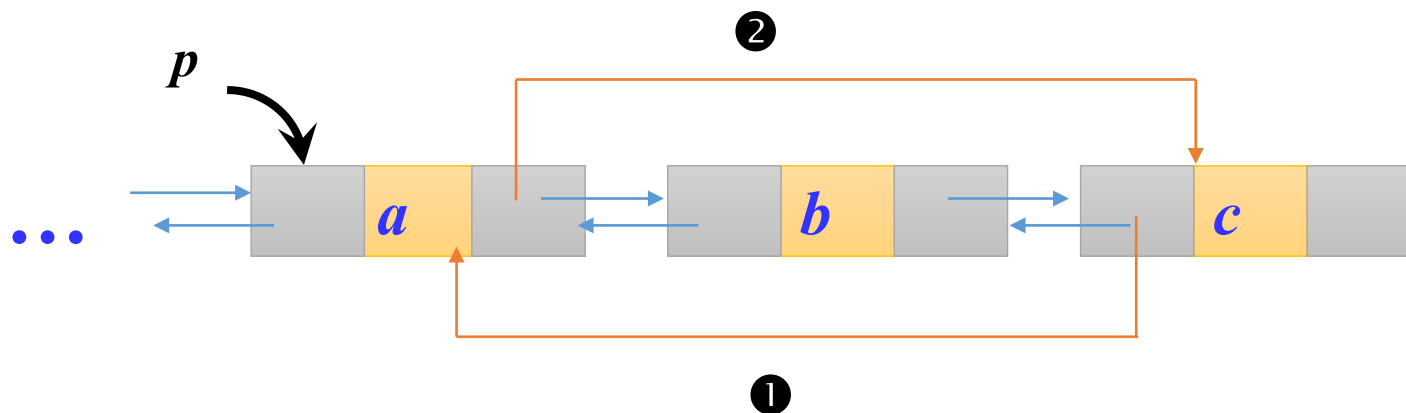
② $p \rightarrow \text{next} \rightarrow \text{prior} = s$

③ $s \rightarrow \text{prior} = p$

④ $p \rightarrow \text{next} = s$ 插入完毕

双向链表的删除

删除 p 结点之后的一个结点



操作语句:

① $p \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{prior} = p$

② $p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next}$

删除完毕

双向链表的建立

整体建立双链表也有两种方法：头插法和尾插法。与单链表的建表算法相似，主要是**插入和删除**的不同。

头插法建立双链表：由含有 n 个元素的数组 a 创建带头结点的双链表 L 。

```
void CreateListF(DLinkNode *&L, ElemType a[], int n)
{ DLinkNode *s; int i;
  L=(DLinkNode *)malloc(sizeof(DLinkNode)); //创建头结点
  L->prior=L->next=NULL;           //前后指针域置为NULL
  for (i=0;i<n;i++)                 //循环建立数据结点
  { s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=a[i];                    //创建数据结点s
    s->next=L->next;                 //将s插入到头结点之后
    if (L->next!=NULL)               //若L存在数据结点，修改前驱指针
      L->next->prior=s;
    L->next=s;
    s->prior=L;
  }
}
```



尾插法建立双链表：由含有 n 个元素的数组 a 创建带头结点的双链表 L 。

```
void CreateListR(DLinkNode *&L, ElemType a[], int n)
{ DLinkNode *s, *r;
  int i;
  L=(DLinkNode *)malloc(sizeof(DLinkNode)); //创建头结点
  r=L;                                       //r始终指向尾结点, 开始时指向头结点
  for (i=0;i<n;i++)                         //循环建立数据结点
  { s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=a[i];                           //创建数据结点s
    r->next=s;s->prior=r;                   //将s插入r之后
    r=s;                                    //r指向尾结点
  }
  r->next=NULL;                             //尾结点next域置为NULL
}
```



双向链表的插入

```
bool ListInsert(DLinkNode *&L, int i, ElemType e)
{ int j=0;
  DLinkNode *p=L, *s;           //p指向头结点, j设置为0
  while (j<i-1 && p!=NULL)       //查找第i-1个结点
  { j++;
    p=p->next;
  }
  if (p==NULL)                   //未找到第i-1个结点, 返回false
    return false;
  else                            //找到第i-1个结点p, 在其后插入新结点s
  {
    s=(DLinkNode *)malloc(sizeof(DLinkNode));
    s->data=e;                    //创建新结点s
    s->next=p->next;              //在p之后插入s结点
    if (p->next!=NULL)            //若存在后继结点, 修改其前驱指针
      p->next->prior=s;
    s->prior=p;
    p->next=s;
    return true;
  }
}
```

另外解法：在双链表中，可以查找第*i*个结点，并在它前面插入一个结点。

双向链表的删除

```
bool ListDelete(DLinkNode *&L, int i, ElemType &e)
{ int j=0;
  DLinkNode *p=L, *q;  //p指向头结点, j设置为0
  while (j<i-1 && p!=NULL)  //查找第i-1个结点
  { j++;
    p=p->next;
  }
  if (p==NULL)  //未找到第i-1个结点
    return false;
  else  //找到第i-1个结点p
  {      //q指向第i个结点
    q=p->next;  //当不存在第i个结点时返回false
    if (q==NULL)
      return false;
    e=q->data;
    p->next=q->next;  //从双向链表中删除q结点
    if (q->next!=NULL)  //若q结点存在后继结点
      q->next->prior=p;  //修改q结点后继结点的前驱指针
    free(q);  //释放q结点
    return true;
  }
}
```

另外解法：在双链表中，可以查找第*i*个结点，并将它删除。

双链表的应用示例

【例2.9】 有一个带头结点的双链表L，设计一个算法将其所有元素**逆置**，即第1个元素变为最后一个元素，第2个元素变为倒数第2个元素，...，最后一个元素变为第1个元素。

- 首先构造一个只有头结点的空双链表
- 为节约空间，可利用原来的头结点和头指针L
- 遍历原链表中剩余结点，采用头插法插到L中

双链表的应用示例

```
void reverse(DLinkNode *&L)    //双链表结点逆置
{ DLinkNode *p=L->next, *q;    //p指向开始结点
  L->next=NULL;                //构造只有头结点的双链表L
  while (p!=NULL)              //扫描L的数据结点
  {                             //用q保存其后继结点
    q=p->next;                  //采用头插法将p结点插入
    p->next=L->next;            //修改其前驱指针
    if (L->next!=NULL)
      L->next->prior=p;
    L->next=p;
    p->prior=L;
    p=q;                        //让p重新指向其后继结点
  }
}
```

线性表 (Linear List) 是数据特性相同的元素构成的有限序列。

线性表

顺序存储结构：顺序表

链式存储结构：链表

单链表

双向链表

循环链表

循环链表



定义：

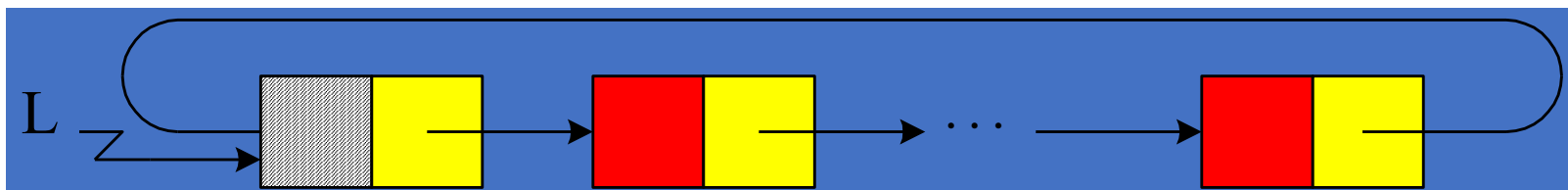
- 是一个首尾相接的链表
- 链表最后一个结点的指针域**不再是空**（ $\wedge$ ），而是指向**头结点**，从而使链表构成一个**环状**。
- 分为：循环单链表和循环双链表

特点：

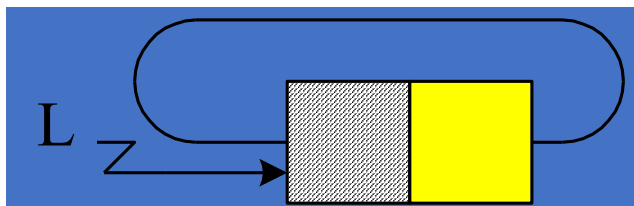
从表中**任何一结点**出发均可以找到表中其他结点，提高查找效率。

循环单链表的结构

与非循环单链表相比，循环单链表没有空指针域：



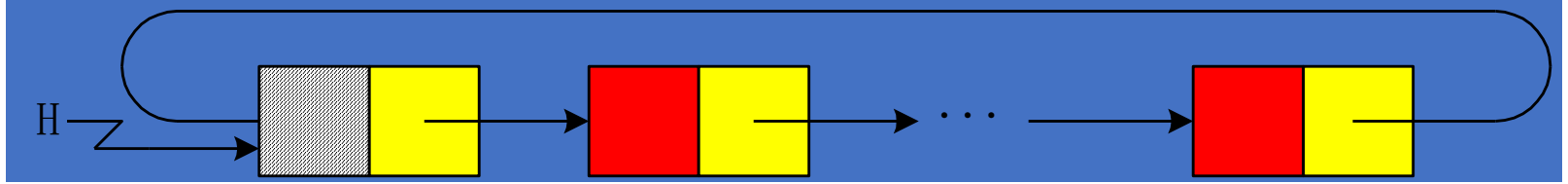
(a) 非空单循环链表



(b) 空表

$p \rightarrow \text{next} = L$

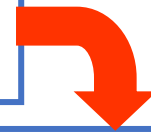
$L \rightarrow \text{next} = L$



从循环链表中的任何一个结点的位置都可以找到其他所有结点，而单链表做不到；



循环链表中没有明显的尾端



如何避免死循环

循环条件： $p \neq \text{NULL} \rightarrow p \neq L$

$p \rightarrow \text{next} \neq \text{NULL} \rightarrow p \rightarrow \text{next} \neq L$

【例2.11】有一个带头结点的循环单链表L，设计一个算法统计其data域值为x的结点个数。

```
int count(LinkNode *L, ElemType x)
{
    int cnt=0;
    LinkNode *p=L->next;           //p指向首元结点,cnt置为0
    while (p!=L)                   //扫描循环单链表L
    {
        if (p->data==x)             //找到值为x的结点后cnt增1
            cnt++;
        p=p->next;                  //p指向下一个结点
    }
    return cnt;                    //返回值为x的结点个数
}
```

循环链表的应用

约瑟夫问题

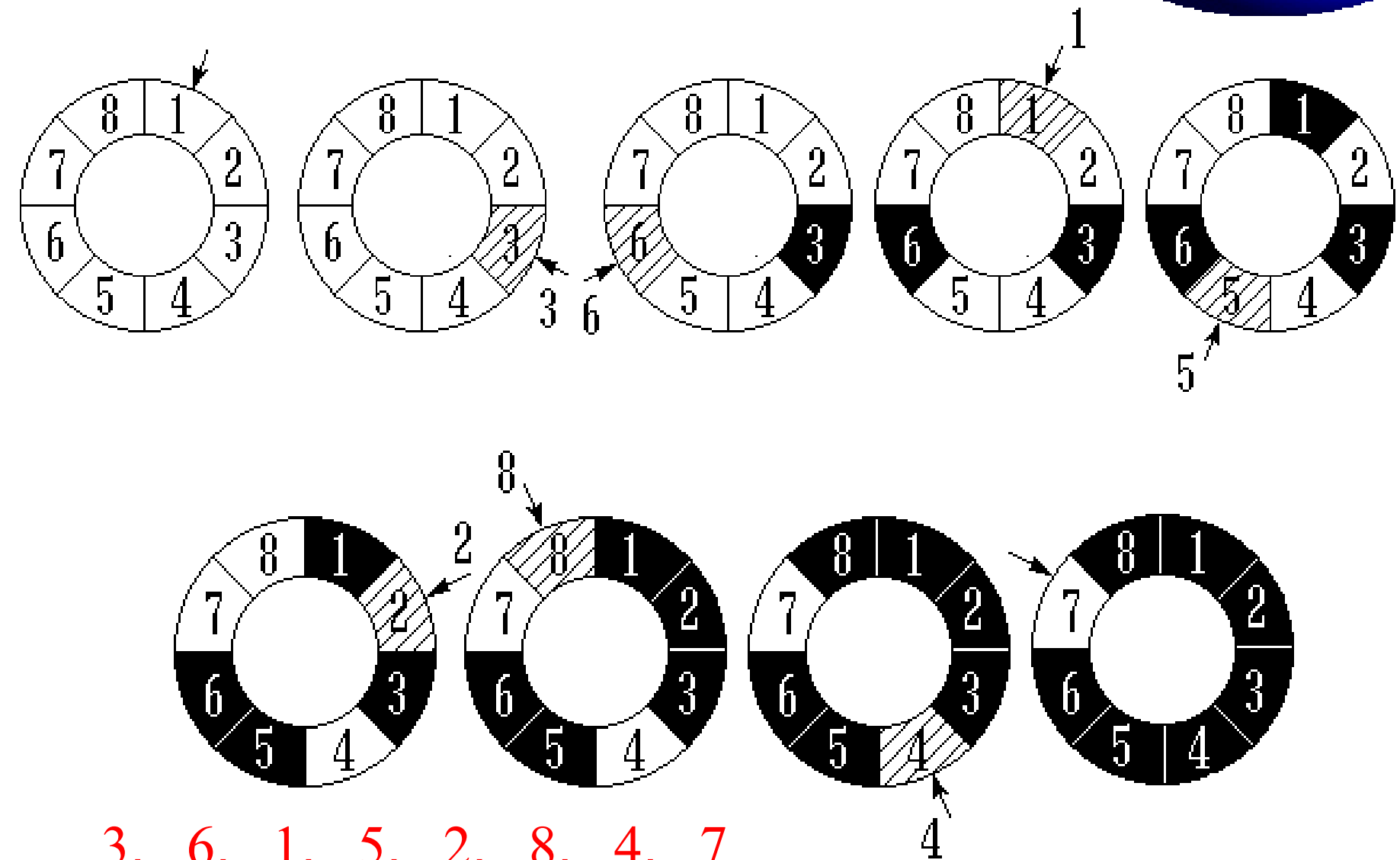
著名犹太历史学家 **Josephus**

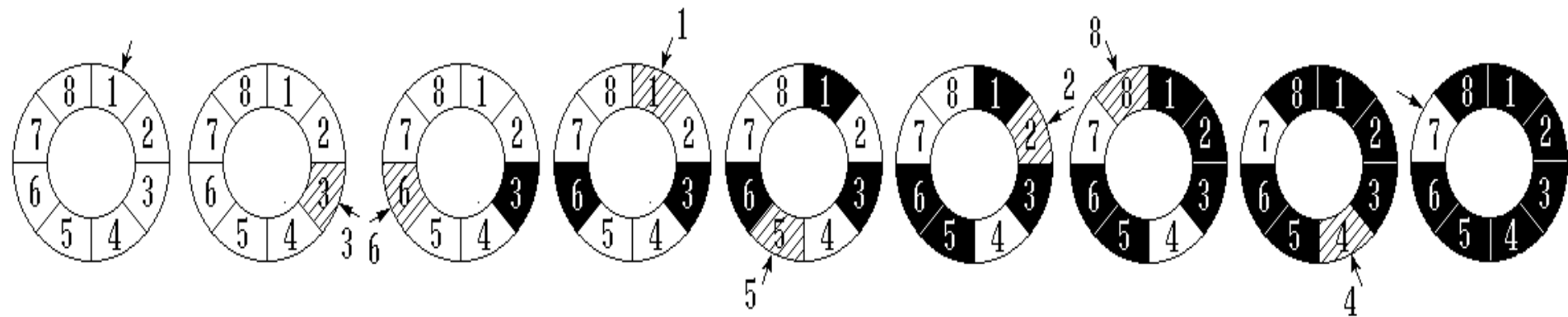
- 在罗马人占领乔塔帕特后
- 39 个犹太人与 Josephus 及他的朋友躲到一个洞中
- 39 个犹太人决定宁愿死也不要被敌人抓到，于是决定了一个自杀方式
- **41 个人(n)**排成一个圆圈，由第 **1(k)** 个人开始报数，每报数到第 **3(m)** 人该人就必须自杀，然后再由下一个重新报数，直到所有人都自杀身亡为止
- 然而 Josephus 和他的朋友并不想遵从，Josephus 要他的朋友先假装遵从，他将朋友与自己安排在第 **16** 个与第 **31** 个位置，于是逃过了这场死亡游戏



约瑟夫问题

• 例如 $n = 8$ $m = 3$ $k = 1$





基本算法思路：

我们将编号为 $1, 2, \dots, n$ 的 n ($n > 0$) 个人围成一个圈表示成一个不带头结点的循环单链表 L ，其中 L 指向第一个结点，每个编号对应一个结点。

假设从第 k 个人开始报数，即从第 k 个结点开始报数，报 m 的人出圈，也就是删除报 m 的结点，再从它的下一结点重新报数，报到 m 时删除结点，如此下去，直到所有结点删除完为止。

(1) 创建一个**不带头结点**的循环单链表

(2) 要求从第 k 个人开始报数，所以还需要有“按序号查找”函数

(3) Josephus算法核心

3, 6, 9, 12, 15, 18, 21,
24, 27, 30, 33, 36, 39, 1,
5, 10, 14, 19, 23, 28, 32,
37, 41, 7, 13, 20, 26, 34,
40, 8, 17, 29, 38, 11, 25,
2, 22, 4, 35, 16, 31

```
51 void Josephus(LinkList L,int k,int n,int m)//核心函数
52 {
53     LNode *s;
54     LNode *t;
55     s=GetLinkList(L,k);//定位到第k个结点
56     cout<<"所有人出队序列如下: ";
57     while(s->next!=s) // 当只剩下一个结点时退出循环
58     {
59         for(int i=1;i<m;i++)
60         {
61             t=s;
62             s=s->next;
63         }
64         t->next=s->next; //删除第m个结点
65         cout<<s->data<<' ';
66         free(s);
67         s=t->next;
68     }
69     cout<<s->data<<endl; // 输出最后一个剩下的结点
70     free(s);
71 }
```

循环双链表

与非循环双链表相比，循环双链表：

- 链表中没有空指针域
- p 所指结点为尾结点的条件： $p \rightarrow \text{next} == L$
- 一步操作即 $L \rightarrow \text{prior}$ 可以找到尾结点



【例2.12】有一个带头结点的循环双链表L，设计一个算法删除第一个data域值为x的结点。

```
bool delelem(DLinkNode *&L, ElemType x)
{ DLinkNode *p=L->next;           //p指向首结点

    while (p!=L && p->data!=x)      //查找第一个data值为x的结点p
        p=p->next;

    if (p!=L)                       //找到了第一个值为x的结点p
    { p->next->prior=p->prior; //删除p结点
      p->prior->next=p->next;
      free(p);
      return true;                 //返回真
    }

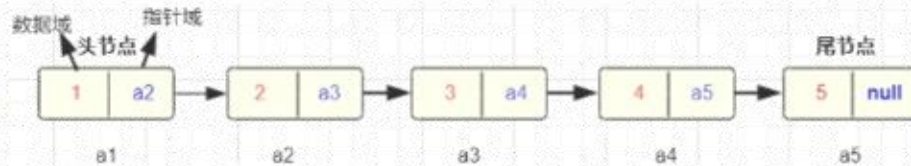
    else                             //没有找到为x的结点，返回假
        return false;
}
```

单链表、循环链表、双向链表

链表

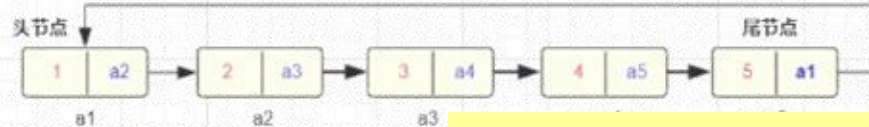
单链表

每个节点保存了下一个节点的引用，只能从头节点往后遍历



循环链表

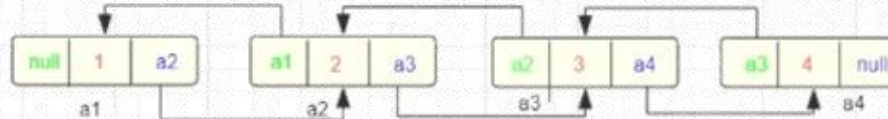
属于一种特殊的链表，循环链表的尾节点链接头节点，形成一个环，这里只列出了单链表的循环链表



双向链表也可以有循环

双链表

每个节点有两个指针域，保存了上一个节点和下一个节点的引用，可以双向遍历



单链表、循环链表、双向链表的时间效率比较

	查找表头结点 (即首元结点)	查找表尾结点	查找结点*p的前驱结点
带头结点的单链表L	L->next 时间复杂度O(1)	从L->next依次向后遍历 时间复杂度O(n)	通过p->next无法找到其前驱
带头结点仅设头指针L的循环单链表	L->next 时间复杂度O(1)	从L->next依次向后遍历 时间复杂度O(n)	通过p->next可以找到其前驱 时间复杂度O(n)
带头结点仅设尾指针R的循环单链表	R->next->next 时间复杂度O(1)	R 时间复杂度O(1)	通过p->next可以找到其前驱 时间复杂度O(n)
带头结点的双向循环链表L	L->next 时间复杂度O(1)	L->prior 时间复杂度O(1)	p->prior 时间复杂度O(1)

线性表的应用



问题描述

两个表自然连接问题

● **表：** m 行、 n 列。假设所有元素为整数。

第1列 第2列 第3列

1	2	3
2	3	3
1	1	1

一个3行3列的表

线性表的应用



问题描述

两个表自然连接问题

- **自然连接** (Natural Join) 是关系数据库中的一种重要运算，它将两个关系（表）在所有公共属性上进行**等值连接**，并在结果中去掉重复的属性列。简单来说，**就是找出两个表中那些在公共列上值相等的行，将它们拼接成一行，并且公共列只保留一份。**
- 举例：学生表

● 两个表自然连接：根据指定列元素是否相等把行拼接起来

<i>A</i>				<i>B</i>	
1	2	3	$\bowtie$ 3=1	3	5
2	3	3		1	6
1	1	1		3	4



$$C = A \bowtie_{3=1} B$$

只有满足 A.列3 = B.列1 的行对才会被保留

<i>C</i>	1	2	3	3	5
	1	2	3	3	4
	2	3	3	3	5
	2	3	3	3	4
	1	1	1	1	6

连接结果



一般格式：

$$C = A \bowtie_{i=j} B$$

数据组织

1	2	3
2	3	3
1	1	1

由于每个表的行数不确定，所以采用单链表作为二维表的存储结构。



3	5
1	6
3	4

每一个行为一个数据节点
(行节点)



注意：头结点和数据结点的类型不同！！！！

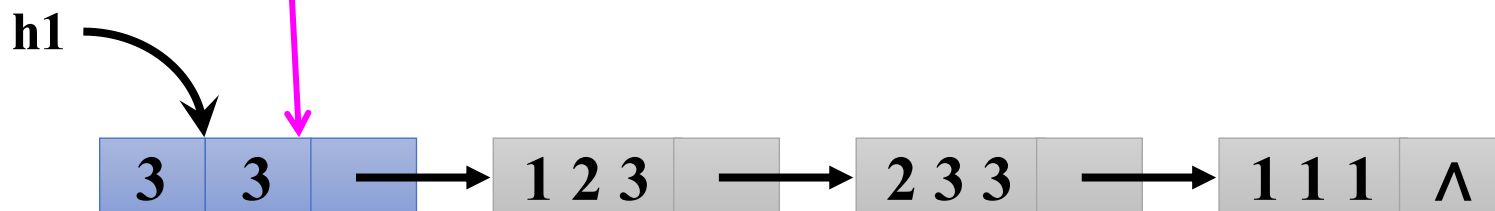
单链表中数据结点类型声明如下：

```
#define MaxCol 10 //最大列数
typedef struct Node1 //定义数据结点类型
{ ElemType data[MaxCol]; //每一行的元素采用顺序存储结构
  struct Node1 *next; //指向后继数据结点
} DList; //行结点类型
```



另外需要指定每个表的行列数，头结点类型声明如下：

```
typedef struct Node2    //定义头结点类型
{ int Row, Col;         //行数和列数，两个数据域
  DList *next;          //指向第一个数据结点
} HList;                //头结点类型
```



顺序表和链表混合使用！！！！

设计基本运算算法

- ① **CreateTable(HList *&h)**: 交互式创建单链表。
- ② **DestroyTable(HList *&h)**: 销毁单链表。
- ③ **DispTable (HList *h)**: 输出单链表。
- ④ **LinkTable(HList *h1, HList *h2, HList *&h)**: 实现两个单链表的自然连接运算。

(1) 交互式创建单链表算法

尾插法

```
void CreateTable(HList *&h)
{ int i, j; DList *r, *s;
  h=(HList *)malloc(sizeof(HList));           //创建头结点
  h->next=NULL;
  printf("表的行数, 列数:");
  scanf("%d%d", &h->Row, &h->Col);           //输入表的行数和列数
  for (i=0;i<h->Row;i++)                       //输入所有行的数据
  { printf(" 第%d行:", i+1);
    s=(DList *)malloc(sizeof(DList));         //创建数据结点
    for (j=0;j<h->Col;j++)                     //输入一行的数据
      scanf("%d", &s->data[j]);

    if (h->next==NULL)                         //插入第一个数据结点
      h->next=s;
    else
      r->next=s;
    r=s;
  }
  r->next=NULL;                               //尾结点next域置空
}
```

采用尾插法建表

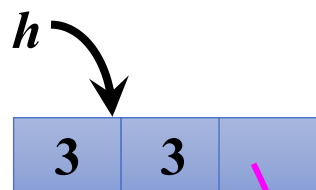
//插入其他数据结点
//将s插入到r结点之后
//r始终指向尾结点

//尾结点next域置空

为什么与一般尾插法建表不一样？

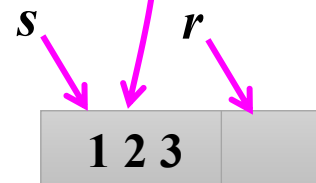
原因是头结点与数据结点类型不同！ r 不能指向头结点。

① 创建头结点 h



② 创建第一个数据结点 s

```
if (h->next==NULL)
{ h->next=s;
  r=s;
}
```



③ 创建其他数据结点 s ，直接链接到 r 结点的后面

```
if (h->next!=NULL)
{ r->next=s;
  r=s;
}
```


(2) 销毁单链表算法

```
void DestroyTable(HList *&h)
{
    DList *pre=h->next, *p=pre->next;
    while (p!=NULL)
    {
        free(pre);
        pre=p;
        p=p->next;
    }
    free(pre);
    free(h);
}
```

(3) 输出单链表算法

```
void DispTable(HList *h)
{ int j;
  DList *p=h->next;           //p指向开始行结点
  while (p!=NULL)             //扫描所有行
  { for (j=0;j<h->Col;j++)    //输出一行的数据
    printf("%4d", p->data[j]);
    printf("\n");
    p=p->next;                 //p指向下一行结点
  }
}
```

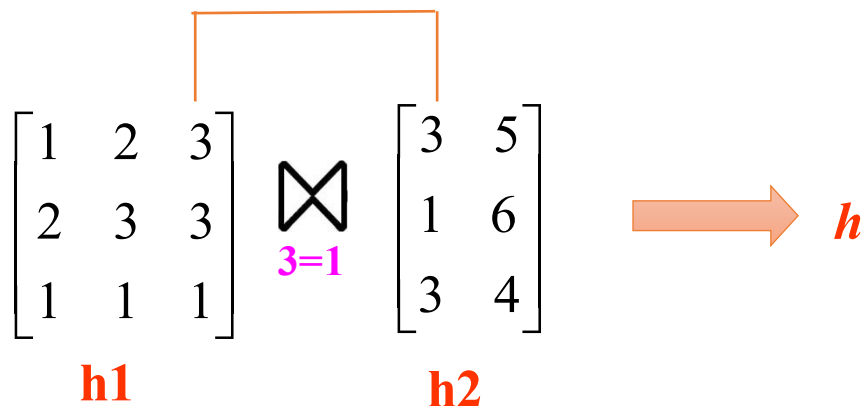
例如，输出一个表：

1	2	3	3	5
1	2	3	3	4
2	3	3	3	5
2	3	3	3	4
1	1	1	1	6

(4) 表连接运算算法

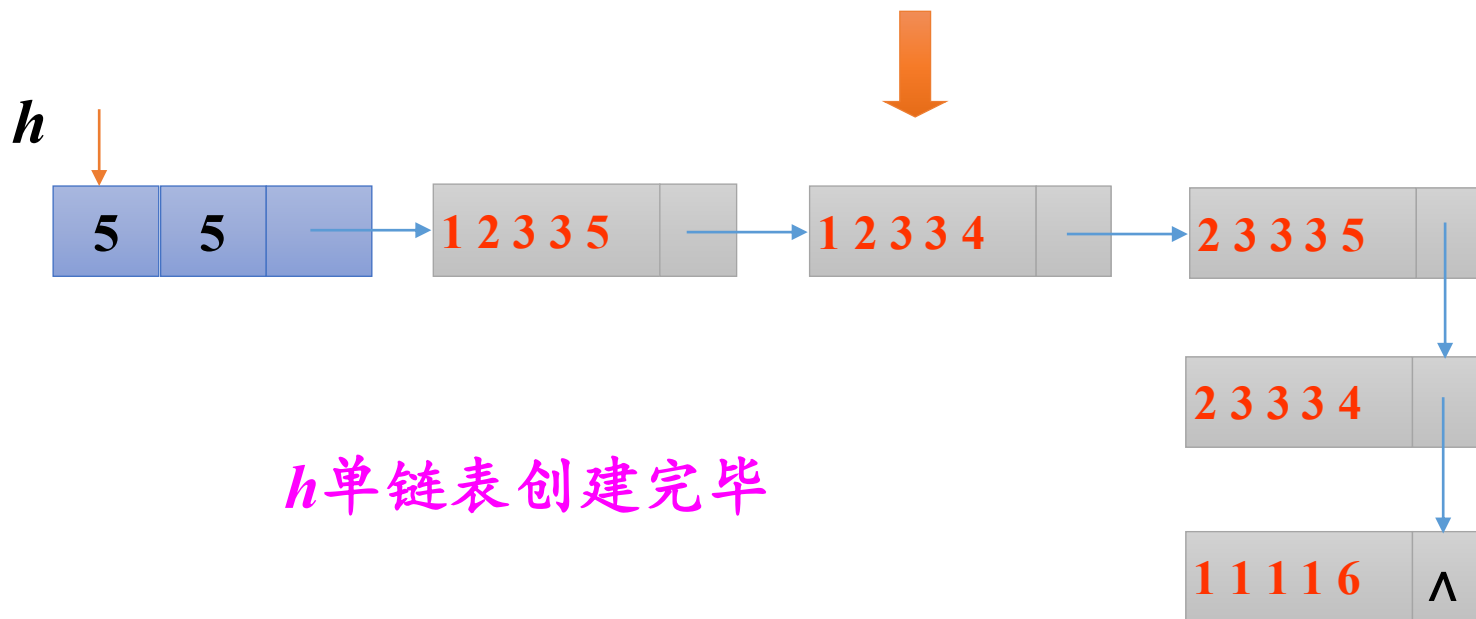
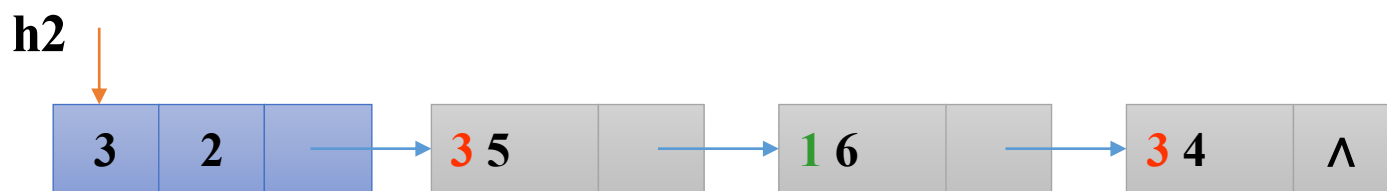
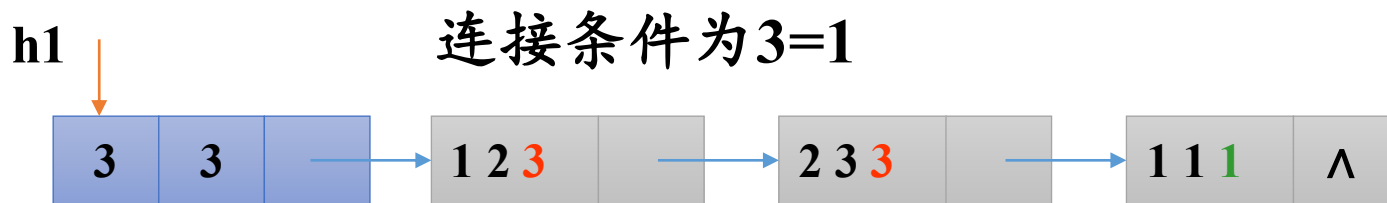
p 扫描 $h1$ 的数据结点, q 扫描 $h2$ 的数据结点。

$$p \rightarrow data[i-1] == q \rightarrow data[j-1]$$



- 对于 $h1$ 的每个数据结点, 都遍历 $h2$ 所有节点
- 一旦条件成立, 就新建一个结点插入到结果单链表 h 中。
- 单链表 h 采用尾插法建表方法创建。

两表条件连接实现的演示



```
void LinkTable(HList *h1, HList *h2, HList *&h)
```

```
{  int i, j, k;
```

```
    DList *p=h1->next, *q, *s, *r;
```

```
    printf("连接字段是:第1个表序号, 第2个表序号:");
```

```
    scanf("%d%d", &i, &j);
```

```
    h=(HList *)malloc(sizeof(HList));
```

```
//创建结果表头结点
```

```
    h->next=NULL;
```

```
//置next域为NULL
```

```
    h->Row=0;
```

```
//置行数为0
```

```
    h->Col=h1->Col+h2->Col;
```

```
//置列数为表1和表2的列数和
```

while (p!=NULL)	//扫描表1
{ q=h2->next;	//q指向表2的开始结点
while (q!=NULL)	//扫描表2
{ if (p->data[i-1]==q->data[j-1])	//对应字段值相等
{	
 s=(DList *)malloc(sizeof(DList));	//创建结点
 for (k=0;k<h1->Col;k++)	//复制表1的当前行
 s->data[k]=p->data[k];	
 for (k=0;k<h2->Col;k++)	//复制表2的当前行
 s->data[h1->Col+k]=q->data[k];	

if (h->next==NULL)	//若插入第一个数据结点
h->next=s;	//将s插入到头结点之后
else	//若插入其他数据结点
r->next=s;	//将s插入到r结点之后
r=s;	//r始终指向尾结点
 h->Row++;	 //表行数增1
}	
q=q->next;	//表2下移一个记录
}	
p=p->next;	//表1下移一个记录
}	
r->next=NULL;	//表尾结点next域置空
}	

求解程序

建立如下主函数调用上述算法：

```
void main()
{ HList *h1, *h2, *h;
  printf("表1:\n");
  CreateTable(h1);           //创建表1
  printf("表2:\n");
  CreateTable(h2);           //创建表2
  LinkTable(h1, h2, h);      //连接两个表
  printf("连接结果表:\n");
  DispTable(h);              //输出连接结果
  DestroyTable(h1);          //销毁单链表h1
  DestroyTable(h2);          //销毁单链表h2
  DestroyTable(h);           //销毁单链表h
}
```


运行结果

表1:

表的行数, 列数:3 3✓

第1行:1 2 3✓

第2行:2 3 3✓

第3行:1 1 1✓

表2:

表的行数, 列数:3 2✓

第1行:3 5✓

第2行:1 6✓

第3行:3 4✓

连接字段是:第1个表位序, 第2个表位序:3 1✓

连接结果表:

1 2 3 3 5

1 2 3 3 4

2 3 3 3 5

2 3 3 3 4

1 1 1 1 6